

Emerging Technologies for the Circular Economy

Lecture 7: IoT Security and Privacy

Prof. Dr. Benjamin Leiding
M.Sc. Anant Sujatanagarjuna
M.Sc. Chintan Patel

License

- This work is licensed under a **Creative Commons Attribution-ShareAlike 4.0 International License**. To view a copy of this license, please refer to <https://creativecommons.org/licenses/by-sa/4.0/> .
- Updated versions of these slides will be available in our [Github repository](#).



SECURING THE IOT

Security

Best Practices

- Most common general security issues:
 - Improper input validation and representation
 - API abuse
 - Missing security features
 - Time and state
 - Poor error handling
 - Poor code quality

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security

Best Practices

- Most common general security issues:
 - Improper input validation and representation
 - API abuse
 - Missing security features
 - Time and state
 - Poor error handling
 - Poor code quality

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most common general security issues:
 - **Improper input validation and representation**
 - API abuse
 - Missing security features
 - Time and state
 - Poor error handling
 - Poor code quality

- Always check that your inputs are valid, e.g. that an email address is actually an email address, and not:
“*abc@asd.com;select * from users;*”
- e.g., SQL Injections & Buffer overflows
- Mitigation: Always allow-list, not deny-list

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - **API abuse**
 - Missing security features
 - Time and state
 - Poor error handling
 - Poor code quality

- Can also be similar to SQL Injection & Buffer overflow attacks
- Attackers commonly like to use scripts or bots to send malicious requests to your APIs to reverse engineer your application
- Mitigation: Proper allow-listing and rate-limiting

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - **Missing security features**
 - Time and state
 - Poor error handling
 - Poor code quality

- “JuSt eNcrYpT eVeRyThiNg”

Just because you use SSL/TLS
Certificates doesn't automatically mean
you are secure from attacks.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - **Missing security features**
 - Time and state
 - Poor error handling
 - Poor code quality

No security through obscurity:
- Proprietary ≠ Secure

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - **Missing security features**
 - Time and state
 - Poor error handling
 - Poor code quality

No security through **visibility**
either:
- FOSS ≠ Secure

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - **Missing security features**
 - Time and state
 - Poor error handling
 - Poor code quality

- “JuSt eNcrYpT eVeRyThiNg”

Just because you use SSL/TLS
Certificates doesn't automatically mean
you are secure from attacks.

- Mitigation: Always prepare for the worst
and dont hope for the best; know where
you are most vulnerable.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - Missing security features
 - **Time and state**
 - Poor error handling
 - Poor code quality

- “Playing with time and state is the biggest untapped natural attack resource.”

- Your (Programmer’s) model of how different processes *should* interact might not be the only possible way an attacker *can* interact with your system.

- Never assume synchronicity of control flows; threads, multi-processor systems, distributed computing are things that exist.

- Mitigation: Asynchronous behavior should ALWAYS be expected: Enforce expected control flows and disallow others.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - Missing security features
 - Time and state
 - **Poor error handling**
 - Poor code quality

- Makes your system easy to break.
- Errors are a great source of “TMI”.
- As with API abuse, common mistakes are to not handle any errors at all, or give out too much information from the stack trace.
- Mitigation: Exhaustive error handling + DEBUG mode $\notin$ production systems.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Security Best Practices

- Most Common general security attack vectors:
 - Improper input validation and representation
 - API abuse
 - Missing security features
 - Time and state
 - Poor error handling
 - **Poor code quality**

- Security $\subset$ Reliability
- If your user experience is unreliable, it can reveal attack vectors to potential attackers.

Mitigation: **Just code better?**

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Privacy

Privacy vs Security

- **Privacy $\neq$ Security**
- Privacy $\not\subset$ Security

Generally speaking:

- Privacy is the “control and proper usage of data”
- Security is the “Protection from unauthorized access.”

Privacy

Privacy vs Security

- Privacy $\neq$ Security
- **Privacy $\not\subset$ Security**

Generally speaking:

Simply because your system is completely secure, doesn't mean you are adhering to all privacy respecting principles.

Privacy

Privacy vs Security

- Privacy $\neq$ Security
- Privacy $\not\subset$ Security
- **Security $\subset$ Privacy**

Generally speaking:

Security is always an essential pre-requisite for having privacy.

Privacy Best Practices

- Most common general privacy issues:
 - **Identification**
 - Localization and Tracking
 - Profiling
 - Lifecycle transitions

- Personally identifiable information is generally always available
- Mitigation: Limit collection, Restrictions on use, retention, and disclosure

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Privacy Best Practices

- Most common general privacy issues:
 - Identification
 - **Localization and Tracking**
 - Profiling
 - Lifecycle transitions

- Determination and recording of the individual's location across space and time.
- Mitigation: Limit collection, Restrictions on use, retention, and disclosure, use accurate location data only when absolutely necessary.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Privacy Best Practices

- Most common general privacy issues:
 - Identification
 - Localization and Tracking
 - **Profiling**
 - Lifecycle transitions

- Compilation of data about users so as to determine their interests through correlation with other sources of data and profiles.

- Profiling could lead to privacy violations if the data is used for unsolicited ads, price discrimination, and social engineering.

- Mitigation: additionally, Transparency: assures all stakeholders that information is managed in accordance with stated commitments and objectives and that its components are visibly transparent.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

Privacy Best Practices

- Most common general privacy issues:
 - Identification
 - Localization and Tracking
 - Profiling
 - **Lifecycle transitions**

- Occurs when used IoT products are sold, used by their owners, and eventually destroyed. It has been established that objects destroy all data, but smart devices often store large amounts of historical data over their lifetime, that are not deleted during the transfer of ownership.

- Mitigation: Strict authentication protocols, data retention policies.

K. Tsipenyuk, B. Chess and G. McGraw, "Seven pernicious kingdoms: a taxonomy of software security errors," in IEEE Security & Privacy, vol. 3, no. 6, pp. 81-84, Nov.-Dec. 2005, doi: 10.1109/MSP.2005.159.

SOME (BAD) EXAMPLES

Who Refuses to Wash Hands?

2012 Seventh International Conference on Broadband, Wireless Computing, Communication and Applications

Who refuses to wash hands?

—

Privacy issues in modern house installation networks

Thomas Mundt

Department of Computer Science
University of Rostock
Rostock, Germany

Email: thomas.mundt@uni-rostock.de

Frank Krüger

Department of Computer Science
University of Rostock
Rostock, Germany

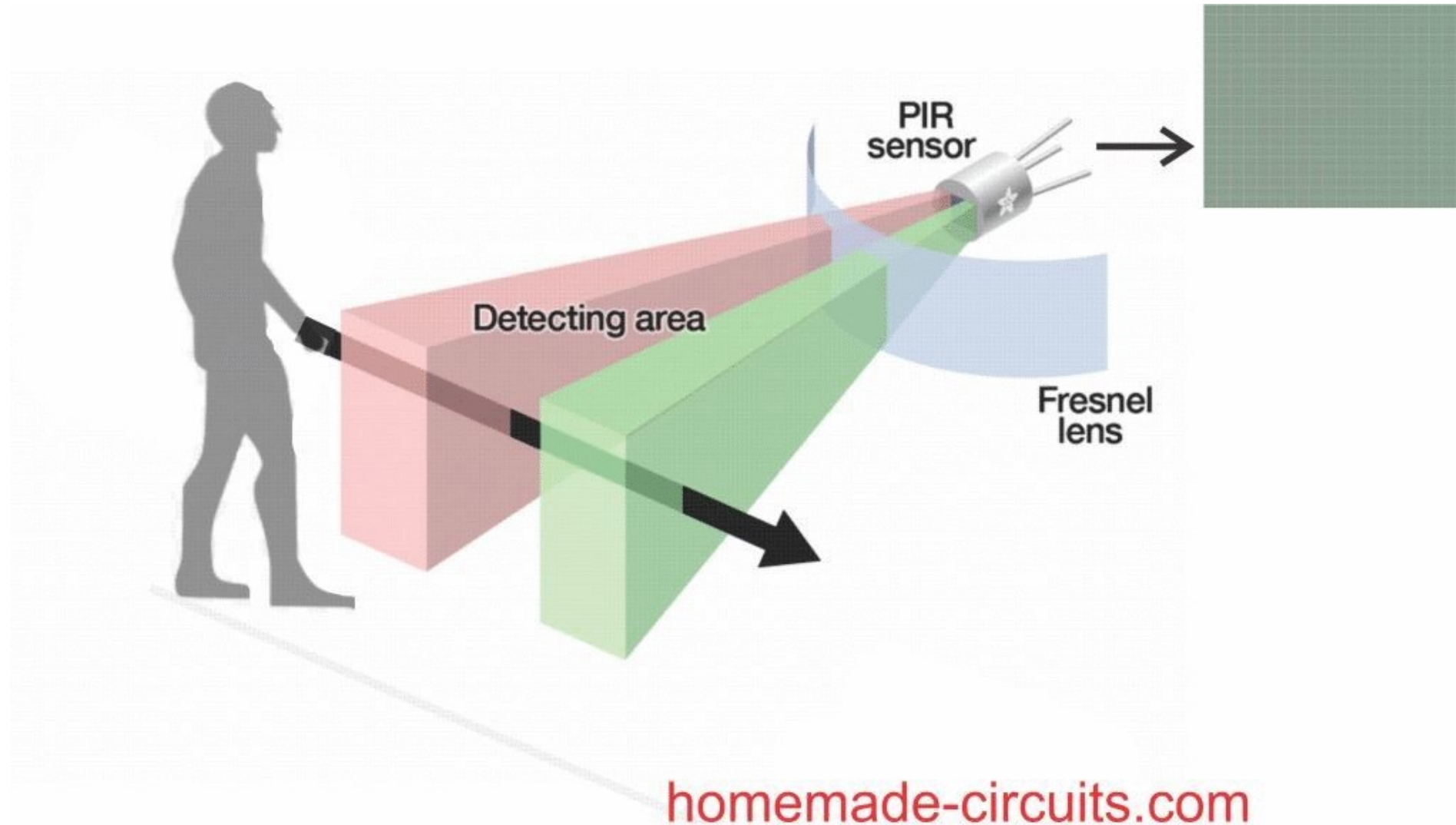
Email: frank.krueger2@uni-rostock.de

Till Wollenberg

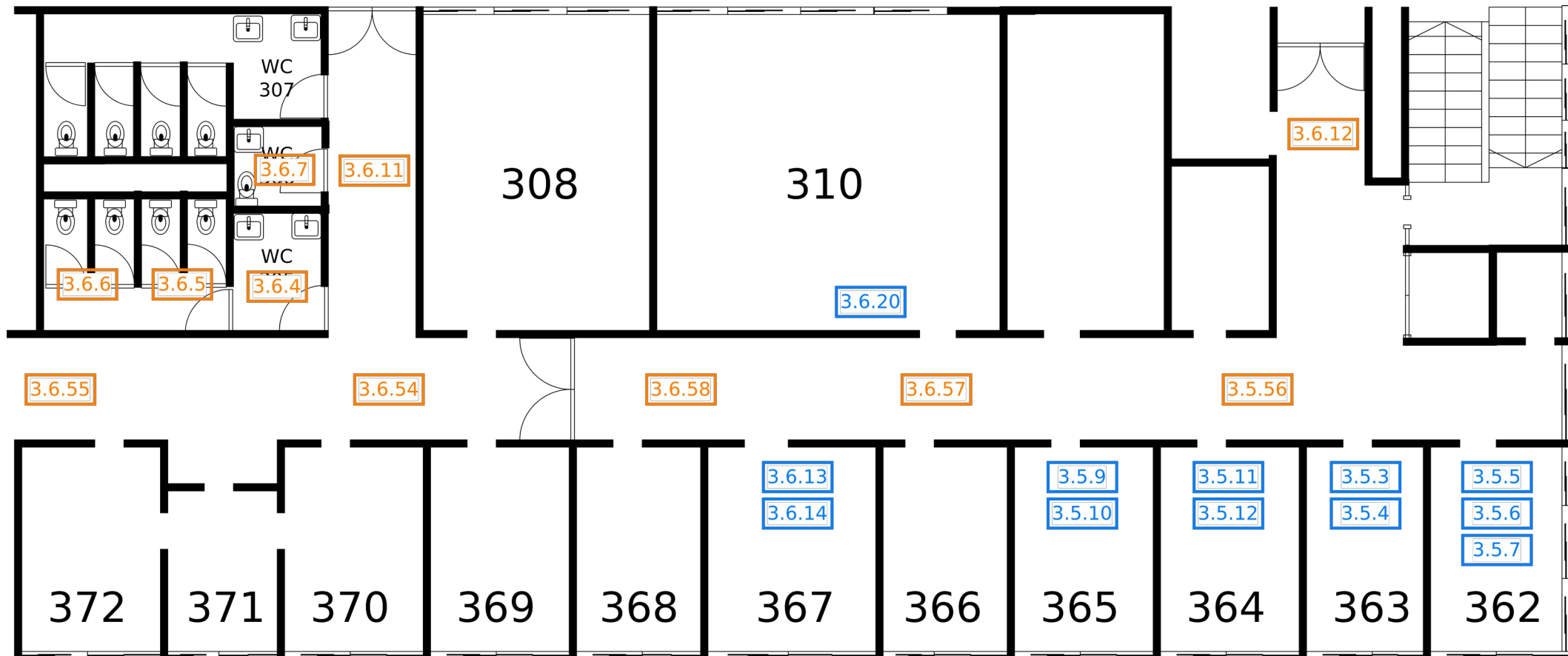
Department of Computer Science
University of Rostock
Rostock, Germany

Email: till.wollenberg@uni-rostock.de

Who Refuses to Wash Hands?



Who Refuses to Wash Hands?



Who Refuses to Wash Hands?

Lessons learned:

- Correlation of data might reveal interesting and unexpected information.
- Implement existing IT security and data protection concepts (KNX can be operated securely) → e.g. encryption and authentication.
- In general:
 - Collect/store/analyze data vs. GDPR
 - Stored data must be secured and protected accordingly

Nissan Leaf Hack (2025)

- Vulnerabilities in the infotainment system's Bluetooth capabilities left the car exposed to privilege escalation.
- Attackers can maintain a stealthy and persistent access to the vehicle remotely over the internet:
 - Doors, vipers, lights, horn, mirrors, etc..
 - Steering wheel!!!!!!
 - Even when the vehicle was in motion



<https://www.securityweek.com/nissan-leaf-hacked-for-remote-spying-physical-takeover/>

Nissan Leaf Hack (2025)

- Vulnerabilities in the infotainment system's Bluetooth capabilities left the car exposed to privilege escalation.
- Attackers can maintain a stealthy and persistent access to the vehicle remotely over the internet:
 - Doors, vipers, lights, horn, mirrors, etc..
 - Steering wheel!!!!!!
 - Even when the vehicle was in motion



“While we decline to disclose specific countermeasures or details for security reasons, for the safety and peace of mind of our customers we will continue to develop and roll out technologies to combat increasingly sophisticated cyberattacks.” -- Nissan

<https://www.securityweek.com/nissan-leaf-hacked-for-remote-spying-physical-takeover/>



IOT SECURITY

IoT Security

Taxonomy of attacks and solutions

Layer/ Component	Attacks	Solutions
Perception Layer		
<i>Perception Nodes (e.g., RFID)</i>	Tracking, DoS, repudiation, spoofing, eavesdropping, counterfeiting, etc..	Access control, data encryption which includes non-linear key algorithms, cryptography techniques to protect against side channel attack, hashed-based access control, etc..
<i>Sensor Nodes</i>	Node subversion, node failure, Node authentication, passive information gathering, sybil, jamming, etc.	Node authentication, Sensor Privacy

IoT Security

Taxonomy of attacks and solutions

Layer/ Component	Attacks	Solutions
Network Layer		
<i>Mobile Communication</i>	Tracking, eavesdropping, DoS, etc.	Secure access control mechanisms, biometrics, public-key cryptography, time-changing session keys.
<i>Cloud Computing</i>	Identity management, data access controls, physical security	• Identity privacy: pseudonymous data store, group signatures • Location privacy: pseudonymous data store, one-way trapdoor permutation • Node compromise attack: Secret sharing, game theoretic mitigation strategies, etc. • Semi-trusted/malicious cloud security: (Fully) homomorphic encryption, zero knowledge proofs.

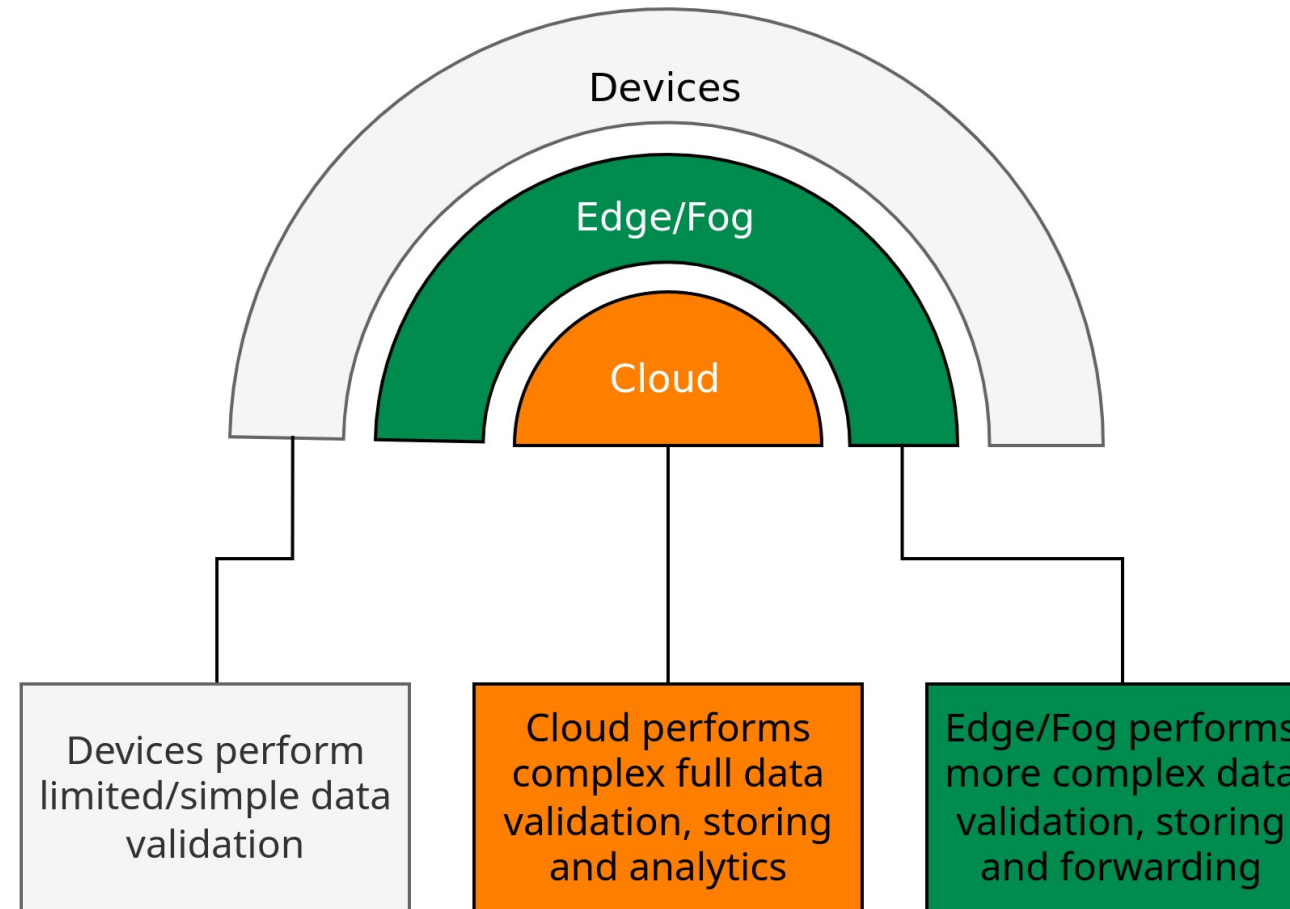
IoT Security

Taxonomy of attacks and solutions

Layer/ Component	Attacks	Solutions
Application Layer	Data privacy, tampering privacy, access control, disclosure of information	Authentication, key agreement and protection of user privacy across heterogeneous networks, end-to-end security, Information flow control.

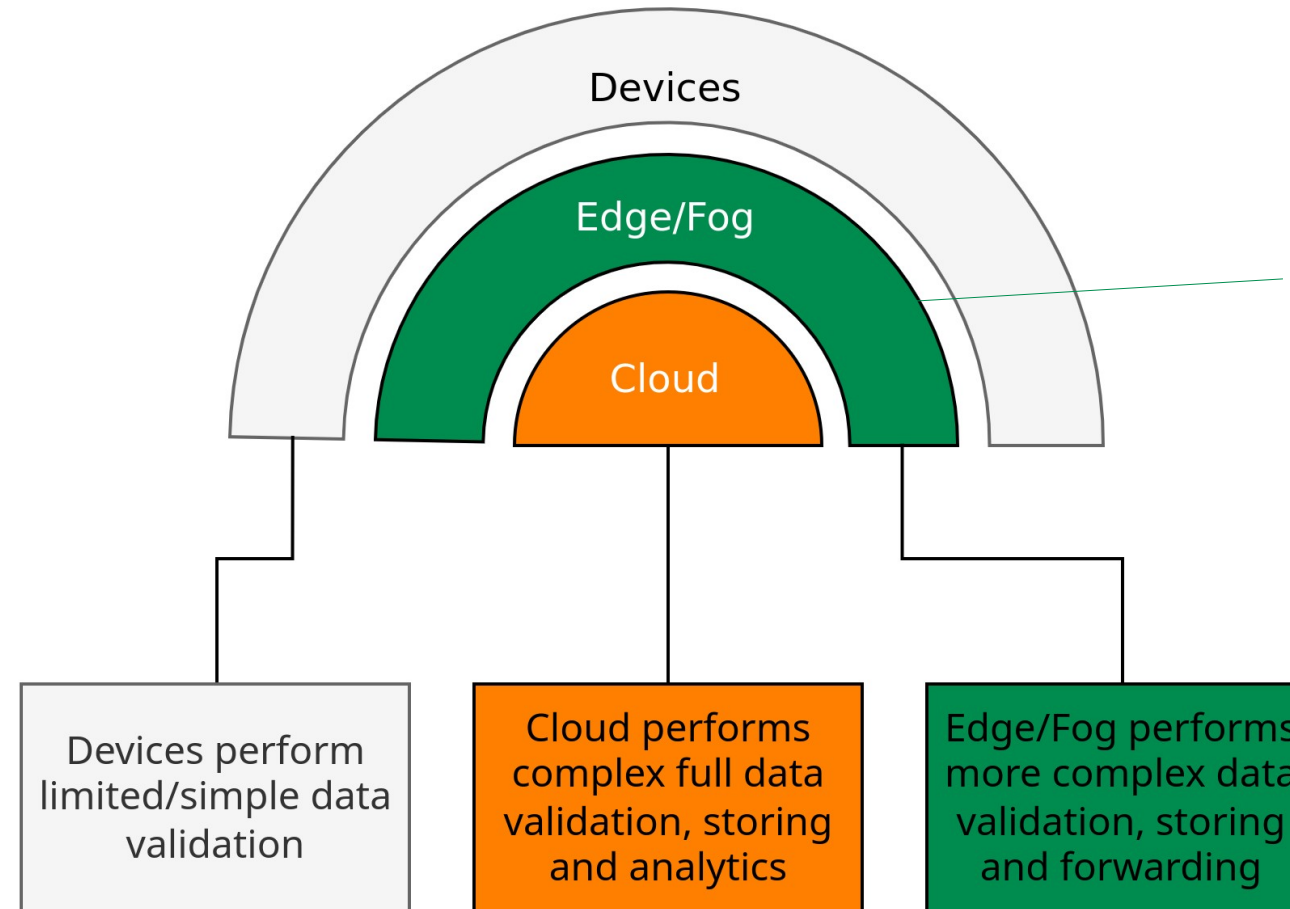
Edge/Fog for IoT (Short detour)

EDGE COMPUTING FOR INTERNET OF THINGS



Edge/Fog for IoT (Short detour)

EDGE COMPUTING FOR INTERNET OF THINGS



Edge/Fog devices need to be secured as well!
- Needs to be handled differently due to lower power/processing power than the cloud

IoT-Edge Security

Edge-based security architecture

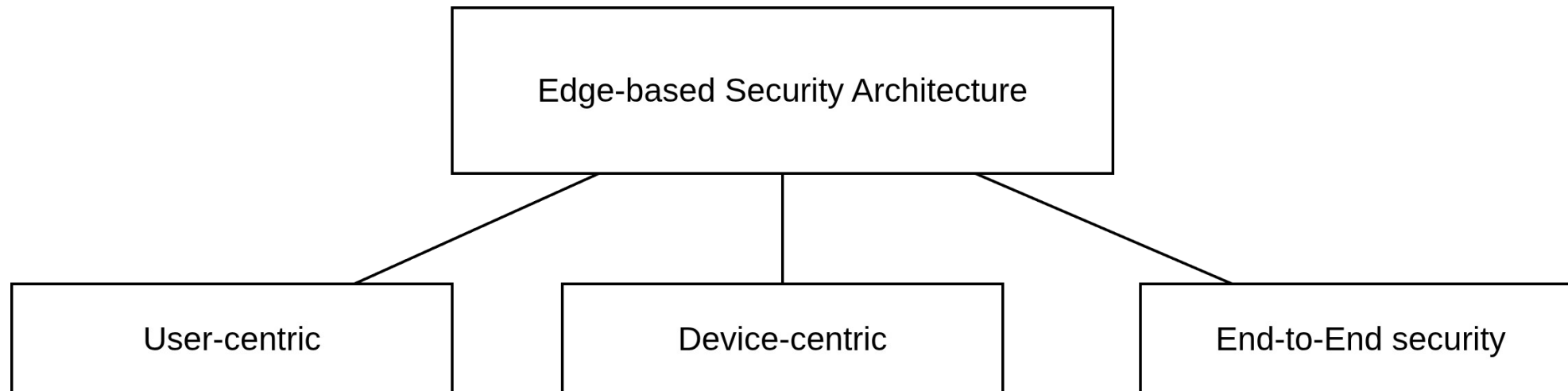


Image recreated from: Kewei Sha, T. Andrew Yang, Wei Wei, Sadegh Davari, A survey of edge computing-based designs for IoT security, Digital Communications and Networks, 2020, <https://doi.org/10.1016/j.dcan.2019.08.006>.

IoT-Edge Security

User Centric Edge-based security architecture

Users are almost always the weakest link in the security architecture. This is especially true for IoT-Edge architectures

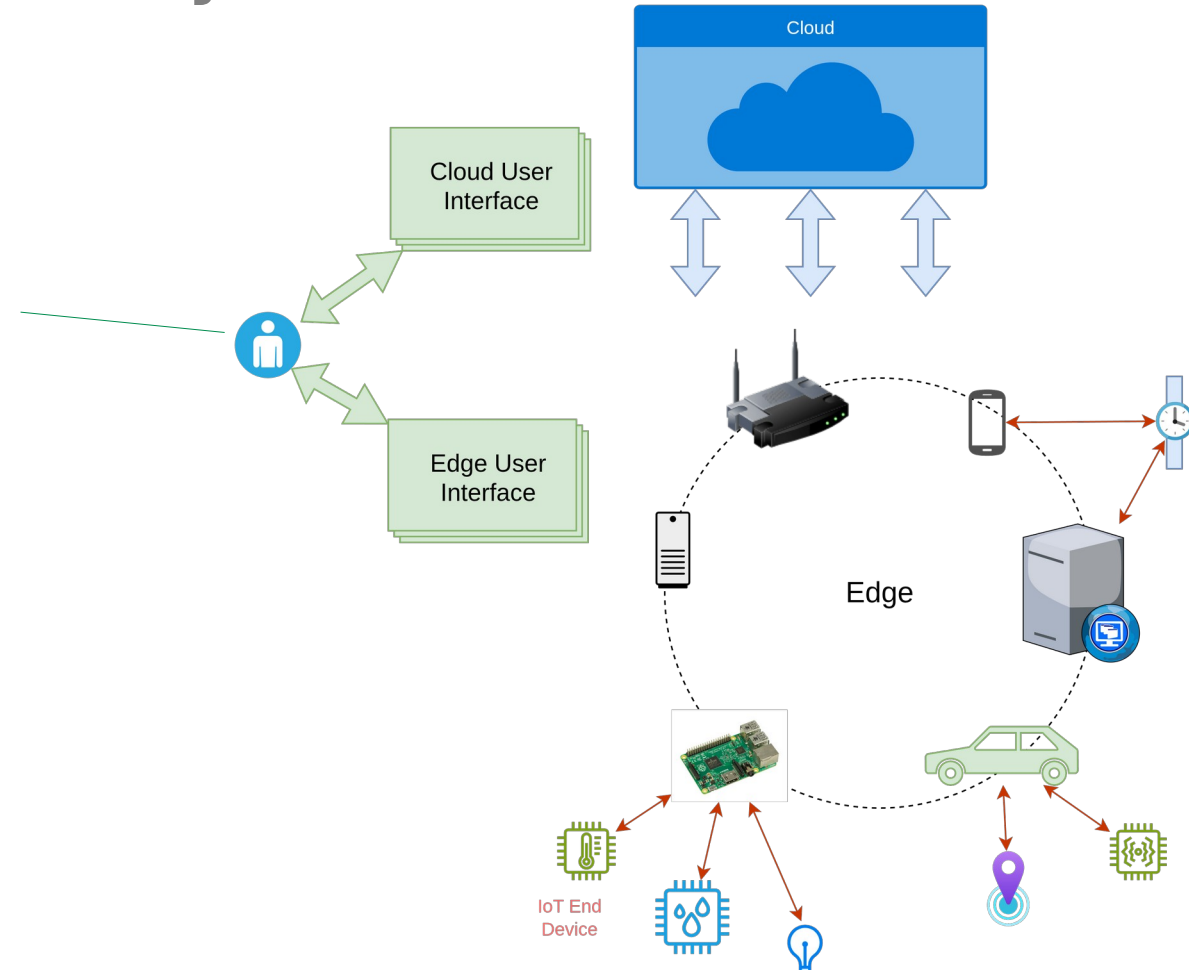
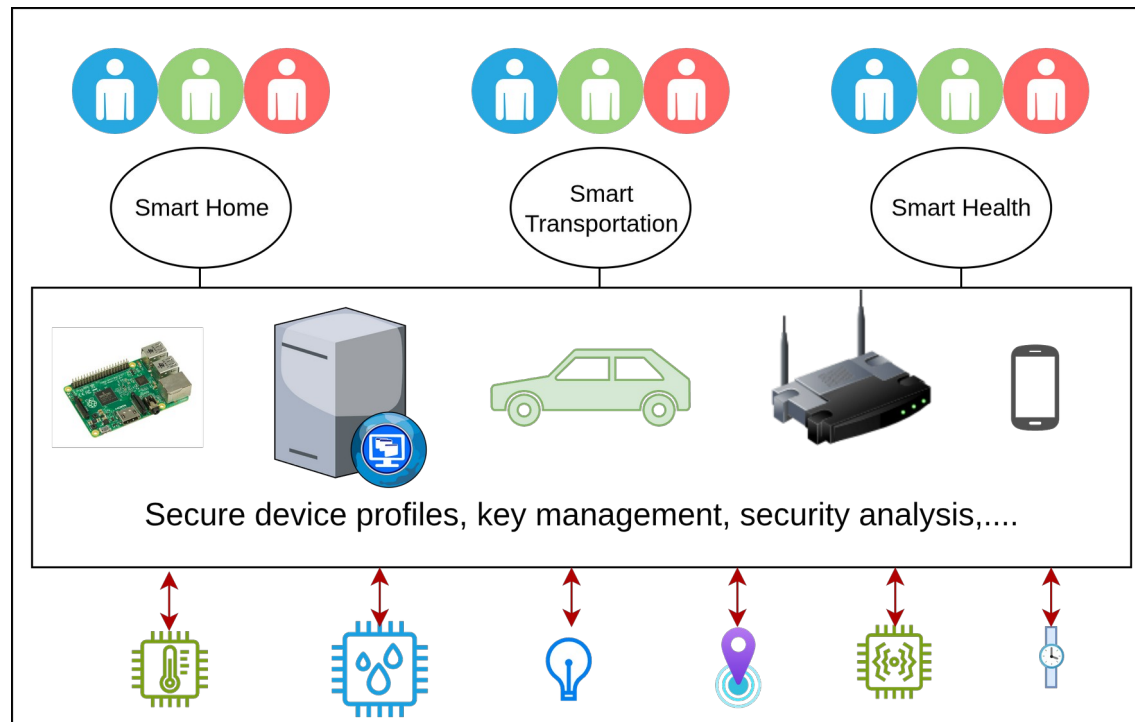


Image recreated from: Kewei Sha, T. Andrew Yang, Wei Wei, Sadegh Davari, A survey of edge computing-based designs for IoT security, Digital Communications and Networks, 2020, <https://doi.org/10.1016/j.dcan.2019.08.006>.

IoT-Edge Security

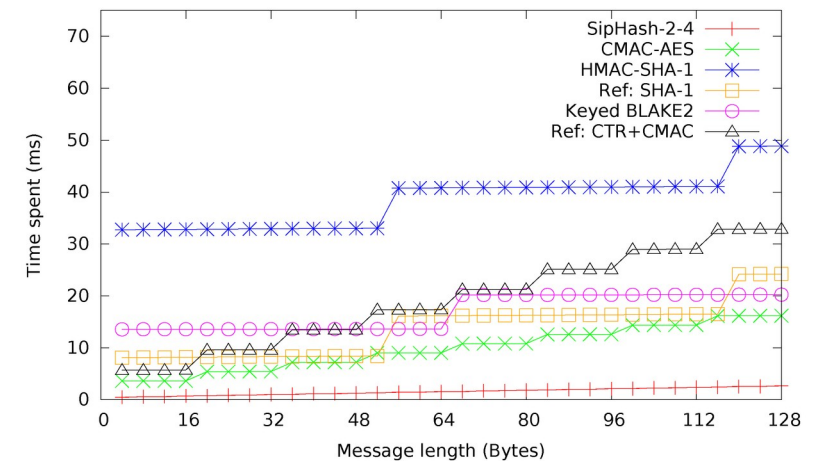
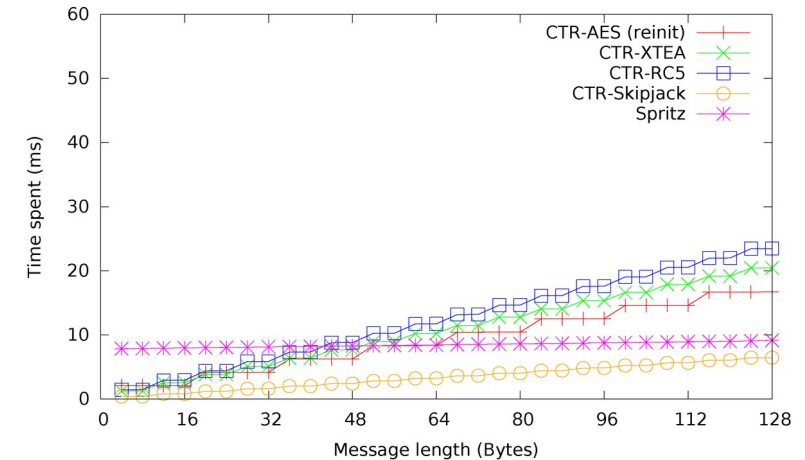
Device Centric Edge-based security architecture



Low power IoT devices cannot use the same security protocols that the cloud uses — we need to use different set of protocols for this.

Cryptography for low power devices

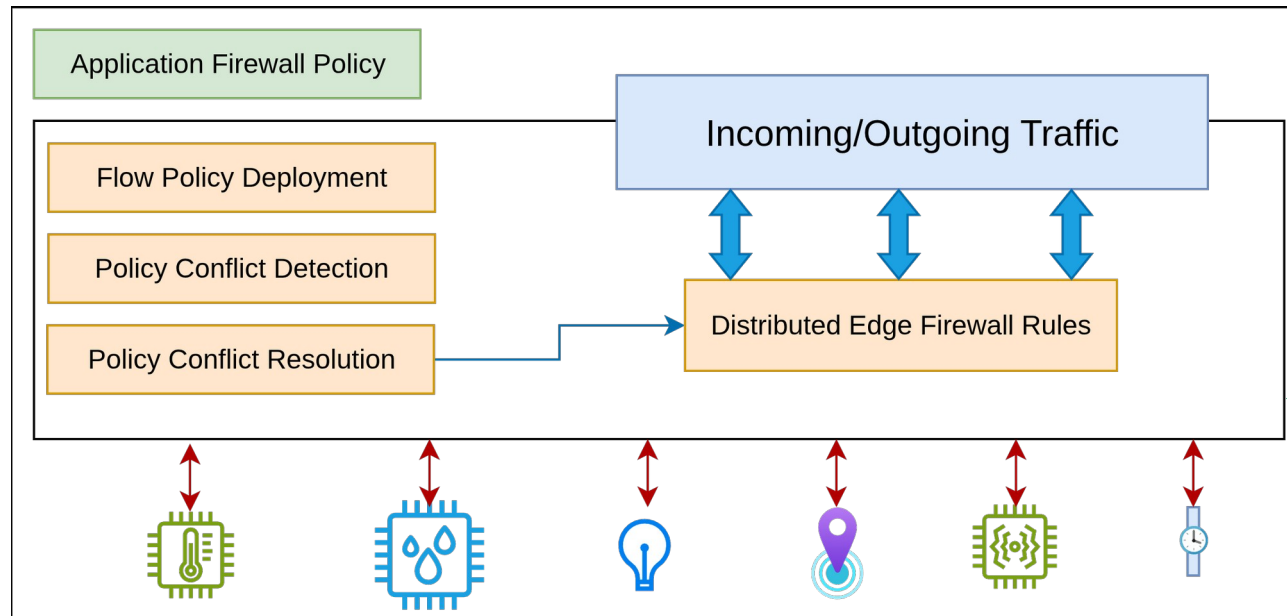
- RSA too heavy for very low-powered devices
- ECC can be acceptable, but still slow
- Main focus on symmetric cryptography:
 - AES128 can perform well even on 8bit micro-controllers
 - Block cipher based MACs
 - SipHash (but is short)
 - SHA family comparatively slow
- Issue: Shared keys



<https://www.mdpi.com/1424-8220/15/8/19560/htm>

IoT-Edge Security

Distributed Edge based Firewalls



Similarly, low power IoT devices cannot easily implement complex and dynamic firewall rules

IoT-Edge Security

Distributed edge-based Intrusion Detection

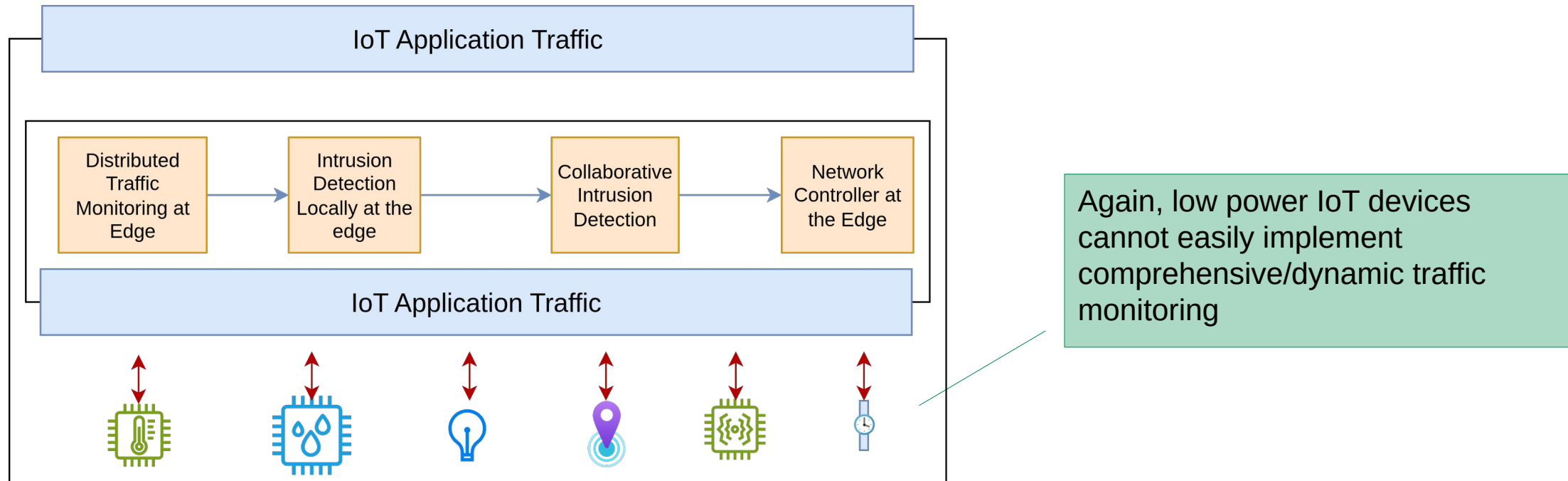
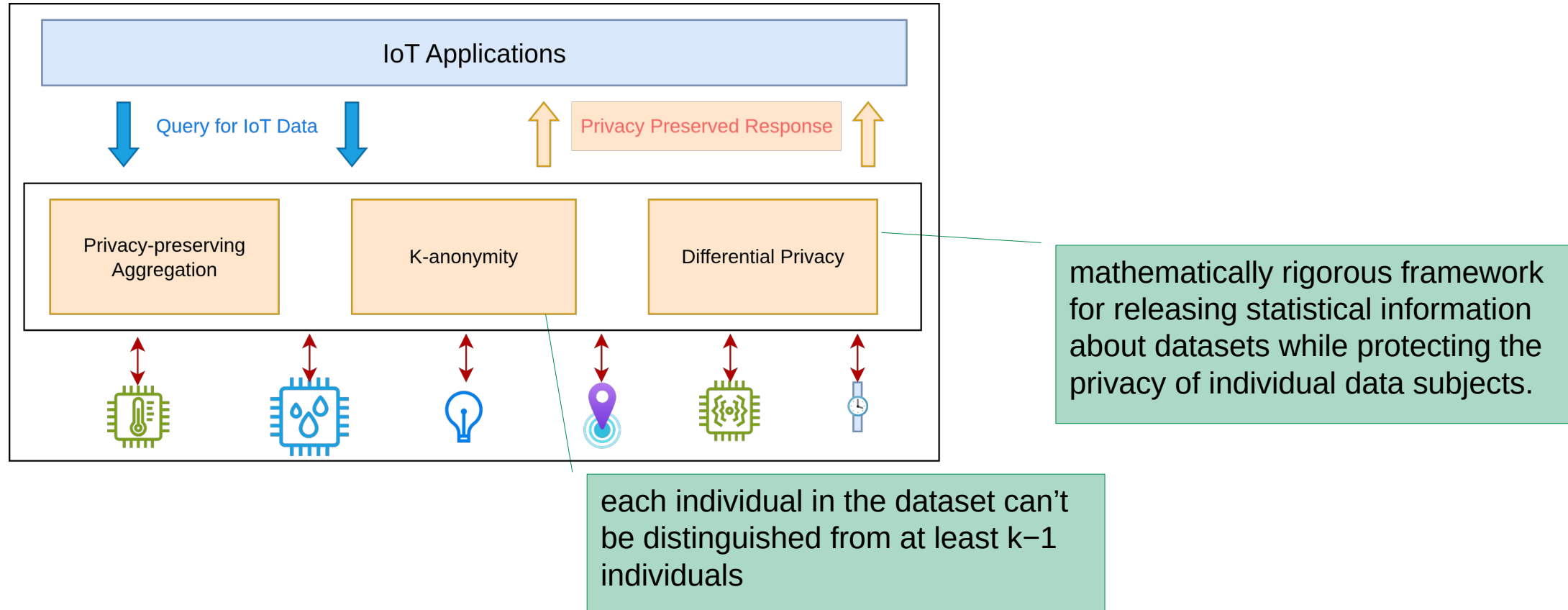


Image recreated from: Kewei Sha, T. Andrew Yang, Wei Wei, Sadegh Davari, A survey of edge computing-based designs for IoT security, Digital Communications and Networks, 2020, <https://doi.org/10.1016/j.dcan.2019.08.006>.

IoT-Edge Privacy

Edge-based privacy preserving design



Summary (1)

- Privacy and Security in IoT is a complex topic
 - Security $\subset$ Privacy
 - Best practices
 - Examples of bad-practices
 - Securing the IoT-cloud architecture vs IoT-Edge architectures

Summary (2)

- General best practices for developing secure software
 - Avoid common issues:
 - Buffer overflows
 - SQL injections
 - etc..
 - Input validation
 - Secure defaults
 - Overall system design
- May also need to consider hardware security
 - Tamper-proof devices
- However: Often IoT devices are not designed with security in mind from the start, they need special attention

Summary (3)

- IoT Privacy is also complex
- Requires:
 - Secure implementations
 - Additional care take to:
 - Minimize personal data being stored/processed
 - Where possible, process locally
 - Where necessary, store encrypted
 - Trust considerations

E06 - IoT Processing

In E05, you gathered weather data from different sources (and sensors) and aggregated them into a single data set. Now, energy sellers will use that data to predict their energy output and sell and serve as many customers as possible. To do so, your task is to use the resulting data set from E05 as an input. The timestamps of the data set begin with 2021-04-10 02:00:00.000; we assume all later timestamps represent a weather prediction in the future.



Questions?